Dossier de conception générale

API REST de gestion d’épargne à objectif

LICELI Technologies

Stage technicien — DIPAMA Gamine — 2026

Sommaire

Sommaire 2

1. Objectif du document 3

2. Choix technologiques 3

3. Modèle de données 4

4. Gestion des erreurs (principes généraux) 6

5. Environnement de déploiement 7

Table des matières 8

1. Objectif du document

Ce document décrit l'architecture globale de la solution : les grands choix techniques, les technologies retenues et leur justification, ainsi que le modèle de données.

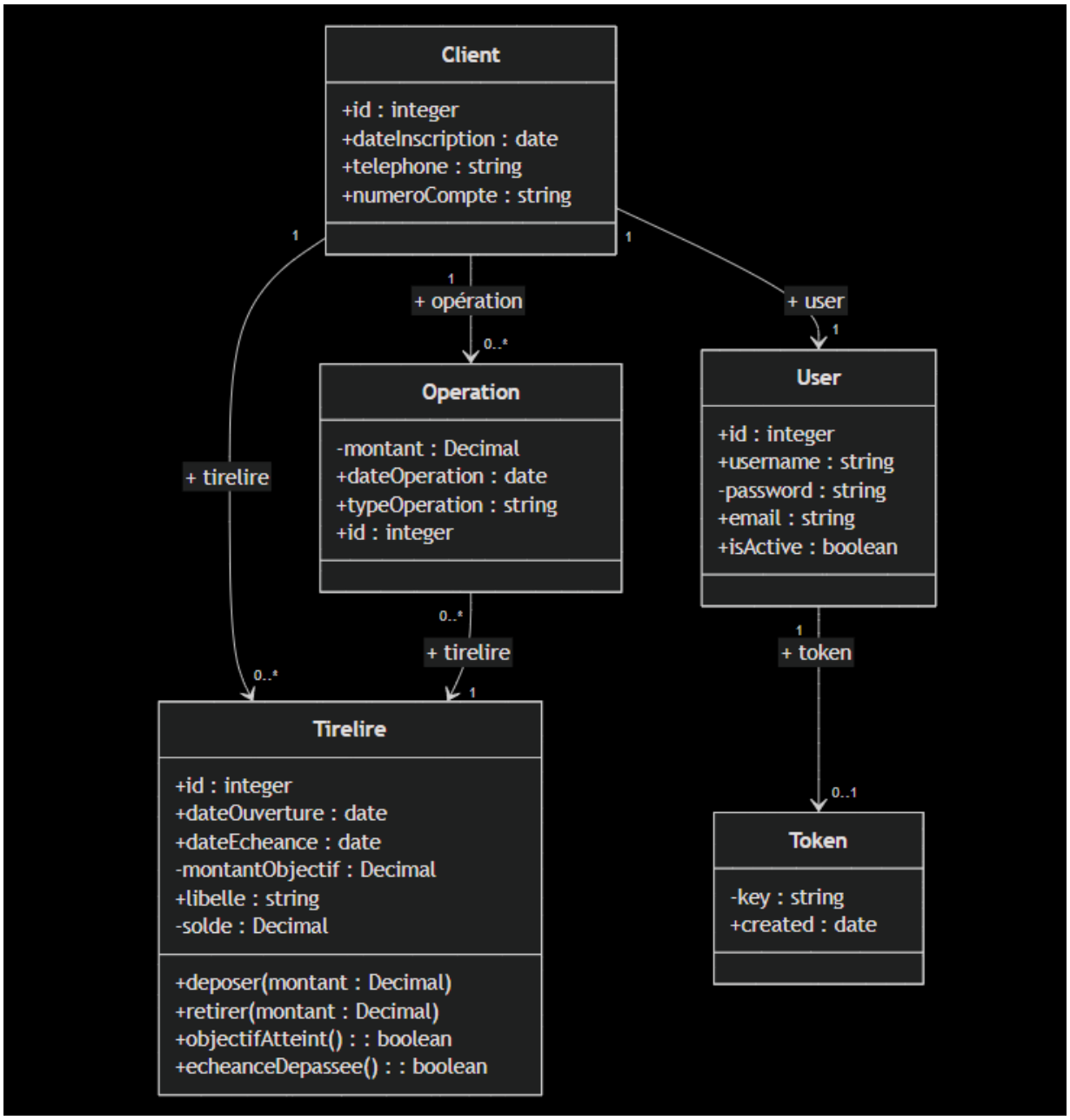
2. Choix technologiques

Élément	Choix retenu	Justification
Langage	Python 3.13	Imposé par le sujet ; écosystème mature pour le développement backend
Framework web	Django 5	Imposé par le sujet ; donne une structure claire (modèles/vues/urls), gère nativement l'authentification, les migrations et l'interface d'administration. Réduit le code à écrire pour se concentrer sur les règles métier
Framework API	Django REST Framework (DRF)	Imposé par le sujet. Extension standard de Django pour exposer des ressources REST ; fournit sérialisation, authentification par token et vues génériques prêtes à l'emploi
Base de données	SQLite (développement)	Imposé par le sujet. Suffisante au regard du volume de données du projet ; aucune installation de serveur de base de données séparé nécessaire, ce qui simplifie l'environnement de développement

Authentification	TokenAuthentication (DRF)	Mécanisme simple, suffisant pour les besoins du projet, sans la complexité additionnelle.
Gestion de version	Git	Imposé par le sujet ; permet un historique de commits lisible, livrable attendu

3. Modèle de données

3.1 Diagramme de classes



3.2 Description des entités

- Client

Représente un utilisateur du système, propriétaire de tirelires. Lié en relation un-à-un avec le modèle `User` natif de Django, qui gère l'identification (nom d'utilisateur, mot de passe haché). Client porte uniquement les informations complémentaires propres au métier de l'épargne (téléphone, numéro de compte, date d'inscription).

- Tirelire

Entité centrale du domaine. Porte les invariants financiers du système (RG1 à RG3) via ses méthodes dédiées. Chaque tirelire appartient à un client unique et peut faire l'objet de plusieurs opérations.

- Opération

Enregistrement historique et immuable d'un mouvement (dépôt ou retrait) sur une tirelire. Une fois créée, une opération n'est jamais modifiée. Elle constitue la traçabilité exigée par les exigences non fonctionnelles du cahier des spécifications fonctionnelles.

- User et Token (fournis par Django et DRF)

L'authentification ne repose sur aucune table créée par le projet : User (Django natif) stocke le nom d'utilisateur et le mot de passe haché ; Token (DRF authtoken) stocke le jeton généré à la connexion, associé à un User unique. Client réutilise User par composition (relation un-à-un) plutôt que de dupliquer ces champs.

3.3 Choix de modélisation justifiés

- Client distinct de User.

Bien que Django fournisse un modèle User suffisant pour l'authentification seule, un modèle Client séparé a été retenu afin de pouvoir porter des informations métier propres (téléphone, numéro de compte) sans alourdir ni détourner le modèle User, et pour permettre une évolution future du profil client sans toucher au mécanisme d'authentification.

- Type Decimal pour tout montant financier (montantObjectif, solde, montant).

Le type float repose sur une représentation binaire approximative des nombres décimaux, produisant des erreurs d'arrondi cumulatives inacceptables sur un produit financier. Le type Decimal garantit une représentation exacte.

- Solde stocké plutôt que recalculé à la volée.

Deux approches étaient possibles : recalculer le solde à partir de la somme des opérations à chaque lecture, ou le stocker directement sur la tirelire et le mettre à jour à chaque opération. Le solde stocké a été retenu pour des raisons de performance : recalculer l'historique complet des opérations à chaque consultation du solde a un coût qui croît avec le nombre d'opérations enregistrées, alors qu'une lecture d'un champ stocké s'effectue en temps constant, quel que soit le volume d'opérations passées. La contrepartie de ce choix qui était le risque de désynchronisation entre le solde stocké et l'historique des opérations est maîtrisée en garantissant que chaque mise à jour du solde s'effectue dans la même transaction atomique que l'enregistrement de l'opération correspondante.

4. Gestion des erreurs (principes généraux)

Conformément aux exigences non fonctionnelles, toute violation d'une règle de gestion doit être interceptée et traduite en réponse HTTP explicite, jamais en erreur serveur non gérée. Le principe retenu :

- Les méthodes métier (sur Tirelire) lèvent des exceptions Python dédiées et nommées (ex. MontantInvalide, SoldeInsuffisant, RetraitNonAutorise).
- Les vues interceptent ces exceptions et les traduisent en réponse HTTP avec un code approprié (400, 404) et un message explicite, conformément au contrat d'API.

Exception	Levée par	Code HTTP	Message retourné
MontantInvalide	deposer(),retirer()	400	Le montant doit être strictement positif.
SoldeInsuffisant	retirer()	400	Solde insuffisant pour ce retrait.
RetraitNonAutorise	retirer()	400	Retrait impossible : objectif non atteint et échéance non arrivée.

DonneesCreationInvalides	Création de tirelire	400	Le montant objectif doit être positif. La date d'échéance doit être future.
TirelireIntrouvable	Toute vue référençant une tirelire par identifiant	404	Tirelire introuvable.
AccesRefuse	Contrôle de cloisonnement	404	Tirelire introuvable. *(même message que ci-dessus, volontairement, pour ne pas révéler l'existence de tirelires appartenant à autrui)*

5. Environnement de déploiement

Le projet est développé et exécuté dans un environnement de développement local (serveur de développement Django, base SQLite). Aucun déploiement en production n'est prévu dans le périmètre de ce stage.

Table des matières

Sommaire 2

1. Objectif du document 3

2. Choix technologiques 3

3. Modèle de données 4

3.1 Diagramme de classes 4

3.2 Description des entités 5

3.3 Choix de modélisation justifiés 5

4. Gestion des erreurs (principes généraux) 6

5. Environnement de déploiement 7

Table des matières 8